

```
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers
```

```
# Гиперпараметры модели
num_classes = 10 # число классов - число цифр
input_shape = (28, 28, 1) # размер изображений цифр, они не цветные, поэтому канал 1.
# загружаем данные (изображения и их классы), отдельно обучающие и тестовые
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data() #
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 — 0s 0us/step
```

```
# это массивы numpy, посмотрите их размеры
print(x_train.shape)
print(y_train.shape)
```

```
(60000, 28, 28)
(60000,)
```

```
# Преобразуем во float и диапазон [0, 1]
x_train = x_train.astype("float32") / 255 #
x_test = x_test.astype("float32") / 255 #
# на всякий случай принудительно сделаем размер (28, 28, 1)
x_train = np.expand_dims(x_train, -1) # для обучающих
x_test = np.expand_dims(x_test, -1) # для тестовых
print("x_train shape:", x_train.shape) # размер массива
print(x_train.shape[0], "train samples") # число примеров обучения
print(x_test.shape[0], "test samples") # число примеров теста
```

```
x_train shape: (60000, 28, 28, 1)
60000 train samples
10000 test samples
```

```
y_train
```

```
array([5, 0, 4, ..., 5, 6, 8], dtype=uint8)
```

```
# переводим метки классов в унитарные вектора
y_train = keras.utils.to_categorical(y_train, num_classes) # обучающие
y_test = keras.utils.to_categorical(y_test, num_classes) # тестовые
y_test[0]
```

```
array([0., 0., 0., 0., 0., 0., 0., 1., 0., 0.])
```

```
print(y_train.shape) # размер массива
print(y_train[0]) # первый элемент его
```

```
(60000, 10)
[0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
```

```
# задаем последовательную модель
model = keras.Sequential( # слои перечисляются ниже
[
    # слой входов (это не нейроны, а именно входы, т.е. данные), указываем размер shape
    keras.Input(shape=input_shape),
    # первый сверточный слой, указываем число фильтров (32), размер ядра (3 на 3), функцию активации (re
    layers.Conv2D(32, kernel_size=(3, 3), activation="relu"),
    # слой пулинга, не обучаемый, но позволит уменьшить размер. указываем размер ядра пулинга (2 на 2)
    layers.MaxPooling2D(pool_size=(2, 2)), #
    layers.Conv2D(64, kernel_size=(3, 3), activation="relu"), #
    # снова пулинг, после этого слоя размер массива выхода уже не большой
    layers.MaxPooling2D(pool_size=(2, 2)), #
    # слой "выпрямления", когда массив вытягивается в вектор, просто изменяет форму массива,
    # нужен чтобы массивы были подходящей размерности.
    layers.Flatten(), #
    # слой дропаута, будет при обучении случайно удалять связи, указываем вероятность удаления связи от
    # layers.Dropout(0.5), #
    # полносвязный слой как в персептроне, размер его равен числу классов, функция активации softmax
```

```

        layers.Dense(num_classes, activation="softmax"), #
    ]
)

```

model.summary() # информация о созданной се

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 10)	16,010

Total params: 34,826 (136.04 KB)
Trainable params: 34,826 (136.04 KB)

```

batch_size = 128 # размер пакета (batch)
epochs = 5 # количество эпох обучения
# задаем функцию ошибки, метод обучения и метрику проверк
model.compile(
    loss="categorical_crossentropy", # кросс-энтропия, одна из возможных функций ошибки, часто используется
    optimizer="adam", # метод обучения Adam - модификация градиентного спуска
    metrics=["accuracy"]) # метрика accuracy
# запускаем обучение ~ 15 минут
model.fit(
    x_train, # входные обучающие данные
    y_train, # желаемые выходные данные (указания учителя)
    batch_size=batch_size, # размер пакета
    epochs=epochs, # максимальное число эпох обучения
    validation_split=0.1) # доля данных, отданных на проверку

```

```

Epoch 1/5
422/422 ————— 8s 10ms/step - accuracy: 0.7941 - loss: 0.7004 - val_accuracy: 0.9767 - val_loss: 0.0835
Epoch 2/5
422/422 ————— 2s 4ms/step - accuracy: 0.9741 - loss: 0.0830 - val_accuracy: 0.9832 - val_loss: 0.0592
Epoch 3/5
422/422 ————— 2s 4ms/step - accuracy: 0.9830 - loss: 0.0581 - val_accuracy: 0.9860 - val_loss: 0.0457
Epoch 4/5
422/422 ————— 2s 4ms/step - accuracy: 0.9853 - loss: 0.0466 - val_accuracy: 0.9880 - val_loss: 0.0442
Epoch 5/5
422/422 ————— 2s 4ms/step - accuracy: 0.9866 - loss: 0.0427 - val_accuracy: 0.9878 - val_loss: 0.0441
<keras.src.callbacks.history.History at 0x7d455752b980>

```

```

score = model.evaluate(x_test, y_test, verbose=0) # оцениваем нашу модель (verbose=0 подавляет вывод на экра
print("Test loss:", score[0]) # функция ошибки на тестовых данных
print("Test accuracy:", score[1]) # метрика (из заданных, у нас accuracy) на тестовых дан

```

Test loss: 0.040294140577316284
Test accuracy: 0.9864000082015991

```

test_example=12 # индекс примера
test_input=x_test[test_example:test_example+1] # изображение этого примера
#test_output=model.predict(test_input)#
# test_output=model.predict_classes(test_input)# распознанный класс для него /больше не работает!!
test_output=np.argmax(model.predict(test_input), axis=-1)
from matplotlib import pyplot as plt #
plt.matshow(test_input[0,:,:,:]) # рисуем матрицу как изображение. Для этой команды нужно подавать двумерный
print('Это цифра',test_output) # выводим номер класса, он же название цифры

```

1/1 0s 441ms/step

Это цифра [9]

